Experiment 3-6 and 8-11

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data = {
    "Glucose": [120, 150, np.nan, 130, 110],
    "BMI": [25.5, 30.2, 28.1, np.nan, 24.0],
    "Age": [25, 45, 35, 50, 30],
    "BloodPressure": [80, 90, 85, 75, np.nan]
}

df = pd.DataFrame(data)

print("Original Dataset:")
print(df)

print("\nMissing Values:")
print(df.isnull().sum())

# Replace missing values with column medians
df = df.fillna(df.median(numeric_only=True))

print("\nCleaned Dataset:")
print(df)

print("\nDescriptive Statistics:")
print(df.describe())

# Visualize glucose distribution
plt.figure(figsize=(7, 4))
sns.histplot(df["Glucose"], kde=True)
plt.title("Distribution of Glucose Levels")
plt.xlabel("Glucose")
plt.ylabel("Frequency")
plt.show()

# Correlation heatmap
plt.figure(figsize=(7, 4))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
plt.title("Healthcare Feature Correlation")
plt.show()
```

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "Age": [19, 24, 28, 32, 37, 42, 48, 55, 61, 68],
    "Income": [20000, 25000, 30000, 35000, 40000,
               45000, 50000, 60000, 70000, 80000],
    "Spending_Score": [20, 35, 40, 55, 60, 65, 70, 75, 85, 95],
    "Purchase_Frequency": [1, 2, 3, 3, 4, 5, 5, 6, 7, 9]
})

print("Original Dataset:")
print(df)

# Equal Width Binning
df["Age_Width_Bin"] = pd.cut(
    df["Age"],
    bins=3,
    labels=["Young", "Middle", "Senior"]
)

# Equal Frequency Binning
df["Income_Frequency_Bin"] = pd.qcut(
    df["Income"],
    q=3,
    labels=["Low", "Medium", "High"]
)

print("\nDiscretized Dataset:")
print(df)

print("\nAge Group Counts:")
print(df["Age_Width_Bin"].value_counts().sort_index())
```

```
print("\nIncome Group Counts:")
print(df["Income_Frequency_Bin"].value_counts().sort_index())

df[["Age", "Income", "Spending_Score",
    "Purchase_Frequency"]].hist(figsize=(10, 7), bins=5)

plt.tight_layout()
plt.show()
```

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

np.random.seed(42)

# Generate synthetic patient data
X = np.random.randint(1, 100, (100, 6))
y = ((X[:, 0] + X[:, 2] + X[:, 3]) > 150).astype(int)

df = pd.DataFrame(X, columns=[
    "Age", "BloodPressure", "Glucose",
    "BMI", "Insulin", "Cholesterol"
])
df["Disease"] = y

print("Dataset:\n", df.head())

X = df.drop("Disease", axis=1)
y = df["Disease"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, pred))
print("Classification Report:\n",
      classification_report(y_test, pred, zero_division=0))

new_patient = scaler.transform([[35, 80, 75, 28, 40, 60]])
print("Predicted Class:", model.predict(new_patient)[0])
```

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

np.random.seed(42)
n = 100

df = pd.DataFrame({
    "Age": np.random.randint(18, 60, n),
    "Gender": np.random.randint(0, 2, n),
    "Location": np.random.randint(0, 3, n),
    "Pages_Visited": np.random.randint(1, 20, n),
    "Time_Spent": np.random.randint(1, 60, n),
    "Product_Views": np.random.randint(1, 15, n),
    "Category_Viewed": np.random.randint(0, 4, n),
    "Search_Count": np.random.randint(0, 10, n)
})

df["Purchase"] = (
    (df["Pages_Visited"] > 10) &
    (df["Time_Spent"] > 25)
).astype(int)

print("Dataset:\n", df.head())
```

```
X = df.drop("Purchase", axis=1)
y = df["Purchase"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = DecisionTreeClassifier(max_depth=4, random_state=42)
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, pred))
print("Classification Report:\n",
      classification_report(y_test, pred, zero_division=0))
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

np.random.seed(42)

df = pd.DataFrame({
    "Monthly_Bill": np.random.randint(200, 2000, 50),
    "Data_Usage": np.random.randint(1, 100, 50),
    "Call_Duration": np.random.randint(10, 500, 50),
    "Recharge_Plans": np.random.randint(1, 10, 50)
})

scaler = StandardScaler()
X = scaler.fit_transform(df)

model = KMeans(n_clusters=2, random_state=42, n_init=10)
df["Cluster"] = model.fit_predict(X)

print("Customer Data:\n", df.head())
print("\nCluster Counts:\n", df["Cluster"].value_counts())
print("\nCluster Averages:\n", df.groupby("Cluster").mean())

plt.scatter(
    df["Monthly_Bill"], df["Data_Usage"],
    c=df["Cluster"]
)
plt.xlabel("Monthly Bill")
plt.ylabel("Data Usage")
plt.title("Telecom Customer Segmentation")
plt.show()
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.preprocessing import StandardScaler

np.random.seed(42)

df = pd.DataFrame({
    "Protein": np.random.randint(10, 100, 10),
    "Gene_Activity": np.random.randint(10, 100, 10),
    "Chemical_Composition": np.random.randint(10, 100, 10)
})

print("Biological Samples:\n", df)

X = StandardScaler().fit_transform(df)

model = AgglomerativeClustering(n_clusters=3, linkage="ward")
df["Cluster"] = model.fit_predict(X)

print("\nCluster Assignments:\n", df)

Z = linkage(X, method="ward")

plt.figure(figsize=(9, 5))
dendrogram(Z)
plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Sample Index")
```

```
plt.ylabel("Distance")
plt.show()
```

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

transactions = [
    ["Milk", "Bread", "Butter"],
    ["Bread", "Butter"],
    ["Milk", "Bread"],
    ["Milk", "Bread", "Butter", "Eggs"],
    ["Bread", "Eggs"],
    ["Milk", "Butter"],
    ["Milk", "Bread", "Butter"],
    ["Bread", "Butter", "Eggs"],
    ["Milk", "Bread", "Eggs"],
    ["Milk", "Bread", "Butter"]
]

# Convert transactions into binary format
te = TransactionEncoder()
encoded = te.fit(transactions).transform(transactions)
df = pd.DataFrame(encoded, columns=te.columns_)

print("Binary Transaction Data:\n", df)

# Find frequent itemsets
frequent = apriori(
    df, min_support=0.2, use_colnames=True
)

print("\nFrequent Itemsets:\n", frequent)

# Generate association rules
rules = association_rules(
    frequent, metric="confidence", min_threshold=0.5
)

print("\nAssociation Rules:")
print(rules[["antecedents", "consequents",
             "support", "confidence", "lift"]])
```

```
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt

# Create a directed graph of web pages
G = nx.DiGraph()

G.add_edges_from([
    ("A", "B"),
    ("A", "C"),
    ("B", "C"),
    ("C", "A"),
    ("D", "C"),
    ("D", "A")
])

# Calculate PageRank
scores = nx.pagerank(G, alpha=0.85)

result = pd.DataFrame(
    scores.items(), columns=["Web Page", "PageRank"]
)

result = result.sort_values(
    by="PageRank", ascending=False
)

print("PageRank Scores:")
print(result.to_string(index=False))

# Visualize the web graph
nx.draw(
    G, with_labels=True, node_size=2000,
    arrows=True
)
plt.title("Web Page Link Structure")
plt.show()
```